

第4回の目標

- よいCSSの考え方を知る
 - 予測しやすい → 他の人がコードを見たときに、記述がどのように影響しているかわかりやすいように書く
 - 再利用しやすい → 使いまわせるパーツのスタイルはまとめる
 - 保守しやすい → 修正・変更した時にわかりやすい書き方にする
 - 拡張しやすい → 追加変更があったときに、最低限のコストで作れるようにする
- CSS設計に基づいた、クラス名のつけ方を知る
 - 原則は英語
 - 基本的な命名規則はBEM
 - ハイフン、接頭辞を正しく使う
- CSS変数を使えるようになる
- リファクタリングをおこなう
 - CSS設計の考え方やルールに基づいて、外部の見た目を変化させずに、内部構造を整理するためにコードを書き換えていく作業
 - CSS設計の命名規則を設定する
 - 違うクラス名だが見た目が同じ要素は、クラス名を統一する
 - カラーコードやコンテンツ幅、z-indexなどをCSS変数を用いて管理する

カスタムプロパティ

- CSS内で「変数」を使った記述の方法。
- よく使用する「値」を変数の中に入れておくことで、何度も同じ記述を書かなくて済む
- 変数を定義するときは「:root (ルートセレクタ)」の中で変数を作る
 - 変数の名前を決めるときは、先頭に「-- (ハイフン)」を2つ添える
- 変数を呼び出すときは、CSSの値の箇所「var()」を使用

以下2つの読み込み結果は同じ

```
.sample {  
  color: #ffbb1e;  
}
```

```
:root {  
  --primary-color: #ffbb1e;  
  --accent-color: #43cfe8;  
}  
  
.sample {  
  color: var(--primary-color);  
}
```

calc関数

- CSSを記述する際に「計算式」を使うことができる関数

使用例

横幅が100%の要素を3等分するようにスタイルを記述するとき

△

```
.sample {  
  width: 33.33333%;  
}
```

◎

```
.sample {  
  width: calc(100% / 3);  
}
```

命名規則

BEM

- Block（ブロック）, Element（エレメント）, Modifier（モディファイア）の頭文字から。
- 書式：**Block**__**Element**__**Modifire**

【BEMで書かない場合のコード】

```
<header class="header">  
  <h1 class="header-logo">  
    ZeroPlus <span class="header-logo-primary">Ad</span>Sense  
  </h1>  
</header>
```

【BEMに直したもの】

```
<header class="header">  
  <h1 class="header_logo">  
    ZeroPlus <span class="header_logo__primary">Ad</span>Sense
```

```
</h1>  
</header>
```

ここでは、

- `Block=header`
- `Element=logo`
- `Modifire=primary`

Blockでよく使うフレーズ

- `nav`
 - ナビゲーションで用いる。
- `post(s)`
 - blogサイトの投稿・記事に用いる。
- `unit(s)`
 - 複数の要素をまとめているものに用いる。
- `list`
 - リストに用いる。
- `item(s)`
 - 複数あるうちの一つとして用いる。
- `btn`
 - ボタンに用いられる。buttonの慣例的な省略方法。
- `kv`
 - ページのファーストビューの画像とキャッチコピーなどをまとめたものに用いる。key visualの慣例的な省略方法。

Elementでよく使うフレーズ

- `cat(s)`
 - カテゴリーに用いられる。categoryの慣例的な省略方法。
- `content(s)`
 - 内容に関する要素を囲う時に用いる。
- `meta`
 - 日付やカテゴリーなどの情報を扱う時に用いる。

- **link**
 - リンクに用いられる。aタグによく使う。
- **title**
 - タイトルに用いられる。単語であることが多い。
- **heading**
 - 見出しに用いられる。文章であることが多い。
- **desc**
 - 説明文や概要の文章に用いられる。descriptionの慣例的な省略方法。
- **txt**
 - テキストに用いられる。抽象度が高いため便利だが注意が必要。
- **thumb**
 - サムネイルに用いる。thumbnailを省略した形。
- **icon**
 - アイコン画像に用いる。
- **logo**
 - ロゴ画像に用いる。
- **img**
 - それ以外の画像に用いる。汎用的で便利。
- **date**
 - 日付に用いられる。
- **copyright**
 - 著作権表記に用いられる。

Modifierでよく使うフレーズ

- **sm**
 - 比較的小さいもの、またはスマホサイズの画面幅の表示に用いられる。smallの慣例的な省略方法。
- **md**
 - 比較的中くらいの大きさ、またはタブレットサイズ以上の画面幅の表示に用いられる。mediumの慣例的な省略方法。
- **lg**

- 比較的大きいもの、またはパソコンサイズ以上の画面幅の表示に用いられる。largeの慣例的な省略方法。
- **primary**
 - サイトで重要なものに用いられる。色や文字によく用いる。
- **sub**
 - サイトで補助的に使用するものに用いられる。

接頭辞

BEMで書いたclass名の前につける文字のこと

接頭辞	役割
l_class-name	汎用的に使えるレイアウトにつかう。sectionレイアウトなど。headerやfooterは便宜上一つのレイアウトとして扱う。
m_class-name	繰り返し汎用的に使えるモジュール（コンポーネントと似たモノ）につかう。
u_class-name	ユーティリティクラス（単一のスタイル・役割を持つclass）につかう。
js_class-name	jsセレクトラにつかう。

- **l** : layout（レイアウト）
 - 共通化できる、大きな「枠」専用のグループ
 - 原則として、**width**や**margin**や**padding**などのようなレイアウトに関わるスタイリングのみ行う。

```
/* コンテンツごとの上下の余白の例 */
.l_contents-wrapper {
  padding: 120px 0;
}

/* コンテンツ幅の例 */
.l_container {
  margin-left: auto;
  margin-right: auto;
  max-width: 1200px;
  width: 92%;
}
```

- **m** : module（モジュール）
 - どこでも使いまわせるパーツのグループ
 - 基本的に、レイアウトに関わるスタイリングは行わない。
 - 拡張性を考えると、レイアウトに関することはMixを使ったり、親要素に依存すると良い

```
<div class="single_main_meta">
  <div class="m_meta">
    <time class="m_meta_date">2021.06.05</time>
    <a href="category.html" class="m_meta_cat">SEO</a>
  </div>
  <!-- /.m_meta -->
</div>
```

```
/* モジュールの例 */
.m_meta {
display: flex;
justify-content: space-between;
align-items: center;
}
.m_meta_date {
color: #aaa;
font-size: 12px;
font-weight: bold;
}
.m_meta_cat {
color: #008db7;
font-size: 12px;
font-weight: bold;
width: 116px;
height: 25px;
display: flex;
align-items: center;
justify-content: center;
border: 1px solid #008db7;
border-radius: 5px;
}

/* レイアウトを分担させている例 */
.single_main_meta {
margin-top: 16px;
}
```

- **u** : utility (ユーティリティ)
 - class名がそのまま、CSSのプロパティ名と値を表現しているもののグループ
 - 名前の付け方はEmmetを参考にすることが多い。
 - Web制作では補助的に使われることがある

例：上に10pxのmarginを付けたい箇所に使い回したいときのクラス名

```
/* 「.u_プロパティ-値」の順で指定している */
.u_mt-10 {
```

```
margin-top: 10px;
}
```

別箇所でする際にも使えるような書き方になっている。

BEM以外に使う命名規則

- 2単語以上をつなげる場合はハイフン (-)
 - ex.) `.class-name`
- モジュール・レイアウト・ユーティリティクラスに関しては接頭辞をつける
 - ex.) `.m_class-name`
- ページ独自のスタイルはページの名字を接頭辞とすることとする。
 - ex.) `.top_block_element__modifier`
- 汎用的に使用可能な名前
 - `wrapper`
 - 意味
 - 包むもの
 - 外側の入れ物
 - 使う場面
 - コンテンツ全体を囲む最も外側の要素
 - 余白や幅の制御をするためのコンテナ
 - よく使う用途
 - ページ全体
 - セクション全体
 - コンポーネント全体
 - `inner`
 - 意味
 - 内側
 - 使う場面
 - `wrapper`の内側に配置し、実際のコンテンツを包む要素
 - 横幅や左右の余白を調整するためのコンテナ
 - よく使う用途
 - セクション内のレイアウト
 - コンテンツの横幅制御
 - 中央寄せ
 - `header`
 - 意味
 - 先頭部分
 - 見出し部分

- 使う場面
 - ページやコンポーネントの上部
 - タイトルやナビゲーションを配置する部分
- よく使う用途
 - ページヘッダー
 - カードヘッダー
 - モーダルヘッダー

- **body**

- 意味
 - 本体
 - メインコンテンツ
- 使う場面
 - コンポーネントの中心となる内容を配置する部分
- よく使う用途
 - 説明文
 - 画像
 - フォーム
 - 一覧

- **footer**

- 意味
 - 末尾部分
 - 終端部分
- 使う場面
 - ページやコンポーネントの下部
 - 補足情報や操作ボタンを配置する部分
- よく使う用途
 - コピーライト
 - ボタン
 - リンク
 - 投稿日時

- 単語を省略する場合

- **cat(s)**

- 正式名称
 - **category**
 - **categories**
- 意味
 - カテゴリー
 - 分類
- 使う場面
 - 記事や商品のカテゴリ分け
- よく使う用途
 - カテゴリー一覧

- タグ表示

- `col`

- 正式名称
 - `column`
- 意味
 - 列
- 使う場面
 - グリッドレイアウト
 - テーブル
- よく使う用途
 - レスポンシブレイアウト
 - 複数カラム表示

- `lv`

- 正式名称
 - `level`
- 意味
 - レベル
 - 階層
- 使う場面
 - メニューや見出しの階層管理
- よく使う用途
 - `menu-lv1`
 - `heading-lv2`

- `desc`

- 正式名称
 - `description`
- 意味
 - 説明
 - 説明文
- 使う場面
 - 商品や記事の概要文
- よく使う用途
 - 商品説明
 - カード説明
 - キャプション

- `btn`

- 正式名称
 - `button`
- 意味
 - ボタン
- 使う場面

- クリック操作を行う要素
- よく使う用途
 - 送信ボタン
 - リンクボタン
 - CTAボタン

- `img`

- 正式名称
 - `image`
- 意味
 - 画像
- 使う場面
 - 画像を表示する要素
- よく使う用途
 - 商品画像
 - アイキャッチ画像
 - プロフィール画像

- `thumb`

- 正式名称
 - `thumbnail`
- 意味
 - サムネイル画像
- 使う場面
 - 一覧表示用の小さい画像
- よく使う用途
 - ブログ一覧
 - 商品一覧
 - 動画一覧

- `num`

- 正式名称
 - `number`
- 意味
 - 数字
 - 番号
- 使う場面
 - 数値を表示する要素
- よく使う用途
 - ランキング
 - ステップ番号
 - 価格表示

- `txt`

- 正式名称

- `text`
- 意味
 - テキスト
 - 本文
- 使う場面
 - 文章を表示する要素
- よく使う用途
 - 本文
 - 説明文
 - コメント
- `sm`
 - 正式名称
 - `small`
 - 意味
 - 小サイズ
 - 使う場面
 - サイズ指定
 - よく使う用途
 - `btn-sm`
 - `text-sm`
- `md`
 - 正式名称
 - `medium`
 - 意味
 - 中サイズ
 - 使う場面
 - サイズ指定
 - よく使う用途
 - `btn-md`
 - `text-md`
- `lg`
 - 正式名称
 - `large`
 - 意味
 - 大サイズ
 - 使う場面
 - サイズ指定
 - よく使う用途
 - `btn-lg`
 - `text-lg`
- `kv`

- 正式名称
 - `key visual`
 - 意味
 - キービジュアル
 - メインビジュアル
 - 使う場面
 - ページのファーストビュー
 - よく使う用途
 - LPのメイン画像
 - コーポレートサイトのトップ画像
- `cta`
- 正式名称
 - `call to action`
 - 意味
 - 行動喚起
 - 使う場面
 - ユーザーに行動を促す要素
 - よく使う用途
 - お問い合わせボタン
 - 資料請求ボタン
 - 購入ボタン
 - 無料登録ボタン

その他意識すること

詳細度をあげないこと

良いCSSを書くためには、セレクタの詳細度をコントロールする必要がある

例えば、次のようなスタイルの指定をむやみに行うと、破綻が起きたり改修段階で不要なコストがかかる。

```
/* 同じ「!important」でしか上書きができなくなる */
.example {
  color: red !important;
}

/* 上書きしづらい + 使いまわせない */
#about {
  /* #aboutに対して与えるスタイル */
  color: red;
}
```

基本的にはclassセレクタで同じ詳細度で書く。

どうしてもidセレクタを使わなければならない時は、次のようにして詳細度を上げることを回避

```
[id='about'] {  
  /* #aboutに対して与えるスタイル */  
  color: red;  
}
```

デザイン依存の命名を避ける

命名方法で、次のように「見た目からつける」というのは避けた方が良い。

```
:root {  
  --yellow-color: yellow;  
  --blue-color: blue;  
}  
  
.class-name__color-red {  
  color: red;  
}
```

このように書いてしまうと、値が変わった時に名前まで変更の必要があり、保守性に欠ける。次のように「意味や役割からつける」方法をとるとよい。

```
/* 変数の命名も役割からつける */  
:root {  
  --primary-color: yellow;  
  --accent-color: blue;  
}  
  
/* 警告用の色として扱う */  
.class-name__warning {  
  color: red;  
}
```